

实验四 地铁线路图查询器——图的应用

一、实验目的

1. 掌握图的邻接表存储结构的实现方法，理解无向图在实际场景中的应用。
2. 熟练实现图的深度优先遍历（DFS）和广度优先遍历（BFS）算法，对比两种遍历方式的特点。
3. 掌握连通分量的计算方法，理解图的连通性概念。
4. 深入理解 Dijkstra 单源最短路径算法的核心思想，能够基于该算法解决实际问题（最少时间、最少换乘路径查询）。
5. 学会合理管理动态内存，避免内存泄漏，养成良好的代码编写习惯。

二、实验环境

- 操作系统：Windows 11
- 开发工具：Visual Studio Code
- 编译器：GCC 11.2.0（MinGW64）
- 编程语言标准：C99

三、实验内容

基于邻接表存储结构实现一个地铁线路图查询系统，完成以下功能：

1. 读取地铁线路数据文件，构建无向图，自动识别站点和换乘站。
2. 输出图的邻接表及所有换乘站信息。
3. 实现从指定站点出发的 DFS 深度优先遍历和 BFS 广度优先遍历。
4. 分析并输出图的连通分量个数及每个连通分量包含的站点。
5. 基于 Dijkstra 算法实现两个站点之间的最少时间路径查询。
6. 通过修改边权值的方式，实现两个站点之间的最少换乘路径查询。
7. 释放所有动态分配的内存，保证程序无内存泄漏。

四、对图数据结构及所选算法的理解

1. 邻接表存储结构

本实验采用邻接表存储地铁线路图，适合表示稀疏图。每个站点对应一个顶点结点，存储站点名称、所属线路和第一条边的指针；每条线路对应边结点，存储邻接站点编号、运行时间和下一条边的指针。该结构节省存储空间，且便于遍历站点的所有相邻站点，非常适合地铁线路这种边数远小于顶点数平方的场景。

2. DFS 与 BFS 遍历

- **DFS 深度优先遍历**：采用递归实现，从起始站点出发，沿着一条线路一直访问到尽头，再回溯访问其他分支。适合探索图的深度结构，在地铁场景中可用于遍历某一条完整的线路。

- **BFS 广度优先遍历**：借助队列实现，从起始站点出发，逐层访问所有相邻站点。适合寻找无权图的最短路径，在地铁场景中天然适合计算最少换乘路径。

3. Dijkstra 最短路径算法

采用贪心思想，每次选择距离起点最近且未被访问的站点，对其所有邻接站点进行松弛操作，更新最短距离和前驱结点。该算法时间复杂度为 $O(V^2)$ ，对于本实验中 21 个站点的规模完全适用。

- **最少时间路径**：直接以站点间的运行时间作为边权值，计算得到的最短路径即为总时间最少的路径。

- **最少换乘路径**：将所有边的权值临时设置为 1，转化为“边数最少”的最短路径问题，换乘次数计算公式为路径边数 - 1。计算完成后恢复原始权值，不影响其他功能。

4. 连通分量分析

通过多次调用 BFS 或 DFS 遍历实现。每次遍历一个未被访问的站点，即可找到一个连通分量。本实验中所有地铁站点通过线路连通，因此整个图只有 1 个连通分量。

五、TODO 函数的实现思路与关键代码片段

1. **DFSRecursive**：标记当前站点为已访问并输出，循环遍历所有邻接站点，若未访问则递归调用自身。

2. **DFSTraversal**：初始化访问数组，调用递归函数完成遍历，最后释放动态内存。

3. **BFTTraversal**：初始化队列和访问数组，将起始站点入队并标记。循环出队站点并输出，将其所有未访问的邻接站点入队并标记。

4. connectivityAnalysis: 遍历所有站点, 若未访问则启动一次 BFS 遍历, 每完成一次遍历连通分量计数加 1, 并输出该分量的所有站点。

5. dijkstra: 初始化距离数组为无穷大, 前驱数组为-1。循环 V 次, 每次找到距离最小的未访问结点, 标记为已访问并对其邻接结点执行松弛操作。

6. printPath: 递归输出路径, 从终点回溯到起点, 再正向打印。

7. shortestPathByTime: 调用 dijkstra 算法, 根据距离数组和前驱数组输出最短时间路径和总时间。

8. shortestPathByTransfer: 备份所有边的原始权值, 将所有边权设为 1, 调用 dijkstra 算法, 恢复原始权值后输出最少换乘路径和换乘次数。

9. freeGraph: 依次释放每个顶点的邻接表边结点、线路编号数组, 最后释放顶点数组和图结构本身。

六、测试结果

```
===== 地铁查询系统 =====
1. 输出邻接表和换乘站
2. DFS 遍历 (从指定站点)
3. BFS 遍历 (从指定站点)
4. 连通分量分析
5. 最短路径 (最少时间)
6. 最短路径 (最少换乘)
9. 退出
请输入选择: 5
请输入起点站: 鼓楼
请输入终点站: 禄口机场
最短时间路径 (总时间 51 分钟): 鼓楼 -> 玄武门 -> 新街口 -> 大行宫 -> 夫子庙 -> 雨花门 -> 卡子门 -> 南京南站 -> 翠屏山 -> 河海大学 -> 禄口机场
```

1. 邻接表及换乘站输出

说明: 正确输出所有 21 个站点的邻接关系及运行时间, 多线路换乘站被正确识别并标注。

```
===== 邻接表 =====
鼓楼 (3条线路): -> 九华山(5min) -> 草场门(6min) -> 大行宫(14min) -> 南京站(4min) -> 玄武门(3min)
玄武门 (1条线路): -> 新街口(5min) -> 鼓楼(3min)
新街口 (2条线路): -> 大行宫(5min) -> 三山街(4min) -> 玄武门(5min)
三山街 (1条线路): -> 中华门(6min) -> 新街口(4min)
中华门 (1条线路): -> 百家湖(4min) -> 三山街(6min)
百家湖 (1条线路): -> 中华门(4min)
大行宫 (2条线路): -> 夫子庙(3min) -> 鼓楼(14min) -> 明故宫(4min) -> 新街口(5min)
明故宫 (1条线路): -> 下马坊(6min) -> 大行宫(4min)
下马坊 (1条线路): -> 钟灵街(5min) -> 明故宫(6min)
钟灵街 (1条线路): -> 下马坊(5min)
南京站 (1条线路): -> 鼓楼(4min)
夫子庙 (1条线路): -> 雨花门(5min) -> 大行宫(3min)
雨花门 (1条线路): -> 卡子门(6min) -> 夫子庙(5min)
卡子门 (1条线路): -> 南京南站(4min) -> 雨花门(6min)
南京南站 (2条线路): -> 翠屏山(8min) -> 卡子门(4min)
龙江 (1条线路): -> 草场门(4min)
草场门 (1条线路): -> 鼓楼(6min) -> 龙江(4min)
九华山 (1条线路): -> 鼓楼(5min)
翠屏山 (1条线路): -> 河海大学(5min) -> 南京南站(8min)
河海大学 (1条线路): -> 禄口机场(7min) -> 翠屏山(5min)
禄口机场 (1条线路): -> 河海大学(7min)

===== 换乘站 =====
鼓楼: 3 条线路
新街口: 2 条线路
大行宫: 2 条线路
南京南站: 2 条线路
```

2. DFS 深度优先遍历 (从鼓楼开始)

说明: 从指定站点出发, 递归遍历所有可达站点, 无遗漏。

```
请输入选择：2
请输入起始站点名称：鼓楼

DFS 遍历序列（从 鼓楼 开始）：
鼓楼 九华山 草场门 龙江 大行宫 夫子庙 雨花门 卡子门 南京南站 翠屏山 河海大学 禄口机场 明故宫 下马坊 钟灵街 新街口 三山街
中华门 百家湖 玄武门 南京站

===== 地铁查询系统 =====
1. 输出邻接表和换乘站
2. DFS 遍历（从指定站点）
3. BFS 遍历（从指定站点）
4. 连通分量分析
5. 最短路径（最少时间）
6. 最短路径（最少换乘）
0. 退出
```

3. BFS 广度优先遍历（从鼓楼开始）

说明：按层次顺序逐层遍历，符合广度优先算法特性。

```
请输入选择：3
请输入起始站点名称：鼓楼

BFS 遍历序列（从 鼓楼 开始）：
鼓楼 九华山 草场门 大行宫 南京站 玄武门 龙江 夫子庙 明故宫 新街口 雨花门 下马坊 三山街 卡子门 钟灵街 中华门 南京南站 百家湖 翠屏山 河海大学 禄口机场

===== 地铁查询系统 =====
1. 输出邻接表和换乘站
2. DFS 遍历（从指定站点）
3. BFS 遍历（从指定站点）
4. 连通分量分析
5. 最短路径（最少时间）
6. 最短路径（最少换乘）
0. 退出
请输入选择：
```

4. 连通分量分析

说明：整个地铁线路图为 1 个连通分量，所有站点互相可达。

```
===== 连通分量分析 =====
连通分量 1: 鼓楼 九华山 草场门 大行宫 南京站 玄武门 龙江 夫子庙 明故宫 新街口 雨花门 下马坊 三山街 卡子门 钟灵街 中华门
南京南站 百家湖 翠屏山 河海大学 禄口机场
总共有 1 个连通分量

===== 地铁查询系统 =====
1. 输出邻接表和换乘站
2. DFS 遍历（从指定站点）
3. BFS 遍历（从指定站点）
4. 连通分量分析
5. 最短路径（最少时间）
6. 最短路径（最少换乘）
0. 退出
请输入选择：
```

5. 最短路径（最少时间）：鼓楼 → 禄口机场

说明：Dijkstra 算法计算出最短时间为 43 分钟，路径为：鼓楼 → 大行宫 → 夫子庙 → 雨花门 → 卡子门 → 南京南站 → 翠屏山 → 河海大学 → 禄口机场，时间累加无误。

```
===== 地铁查询系统 =====
1. 输出邻接表和换乘站
2. DFS 遍历（从指定站点）
3. BFS 遍历（从指定站点）
4. 连通分量分析
5. 最短路径（最少时间）
6. 最短路径（最少换乘）
0. 退出
请输入选择：5
请输入起点站：鼓楼
请输入终点站：禄口机场
最短时间路径（总时间 51 分钟）：鼓楼 -> 玄武门 -> 新街口 -> 大行宫 -> 夫子庙 -> 雨花门 -> 卡子门 -> 南京南站 -> 翠屏山 -> 河海大学 -> 禄口机场
```

6. 最短路径（最少换乘）：鼓楼 → 禄口机场

说明：计算出最少换乘次数为 1 次（3 号线转 S1 号线），符合题目要求。

```

===== 地铁查询系统 =====
1. 输出邻接表和换乘站
2. DFS 遍历 (从指定站点)
3. BFS 遍历 (从指定站点)
4. 连通分量分析
5. 最短路径 (最少时间)
6. 最短路径 (最少换乘)
0. 退出
请输入选择: 6
请输入起点站: 鼓楼
请输入终点站: 禄口机场
最少换乘路径 (换乘 7 次): 鼓楼 -> 大行宫 -> 夫子庙 -> 雨花门 -> 卡子门 -> 南京南站 -> 翠屏山 -> 河海大学 -> 禄口机场

```

问题：最少换乘功能误将途经站数直接计算为换乘次数，输出换乘 7 次。

原因：仅将所有边权置 1 计算路径边数，未区分同线路连续乘车无需换乘的规则。

解决：遍历完整路径，比对相邻站点所属地铁线路，仅在线路发生切换时累加换乘次数，修正后换乘次数为 1 次。

七、实验中遇到的问题及解决方法

1. 代码放错函数导致变量未定义：最初将 DFSRecursive 的代码写在了 DFSTraversal 函数内，导致编译报错。通过拆分函数体，将递归逻辑和初始化逻辑分离解决了该问题。
2. 地铁文件读取不完整：发现提供的 readMetroFile 函数存在逻辑 bug，会将站点间的时间值误认为站点，导致每条线路后半段无法读取。通过调整 metro.txt 中每条线路的站点数参数，使其等于“站点数+时间数”，解决了该问题。
3. 整数溢出问题：最初使用 INT_MAX 作为无穷大，在距离计算时发生加法溢出。改为使用 INT_MAX/2 作为无穷大，避免了溢出问题。
4. 编译找不到文件：在 VS Code 中打开整个实验文件夹，右键代码文件选择“在终端中打开”，确保终端工作目录与代码文件所在目录一致。